

Verida Token (VDA)

SMART CONTRACT

Security Audit

Performed on Contracts:

VDA-V1.sol
VDA-Lock.sol
RecipientLockUp.sol
IVDA-Lock.sol
IVDA.sol
Mock-VDA-V2.sol
IRecipientLockUp.sol
VDA-V1.sol (interface collection)

Platform

ETH

hashlock.com.au

FEBRUARY 2023

Table of Contents

Executive Summary	4
Project Context	4
Verida's Executive Team Leadership	7
Audit scope	8
Security Rating	9
Standardised Checks	10
Intended Smart Contract Functions	12
Function List	14
VDA-Lock.sol Function Analysis	19
Code Quality	22
Audit Resources	22
Dependencies	22
Severity Definitions	23
Audit Findings	24
Centralisation	33
Owner executable functions	34
Conclusion	35
Our Methodology	36
Disclaimers	38
About Hashlock	39
Appendix	40

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Verida's team partnered with Hashlock to conduct a security audit of their ethereum smart contracts prior to their project's launch. Hashlock manually and proactively reviewed the code in order to ensure the protocols team and community is ready for mainnet deployment.

This audit is the first out of two planned audits we will be conducting for Verida, and at the time of this report being written, the entire Verida network protocol has not been audited by Hashlock as of yet.

Project Context

[Verida Network](#) is a multi-chain protocol for interoperable decentralised data storage, communications infrastructure and authentication solutions built around the fundamentals of individual ownership of data and identity. This network can be interfaced with end users via their mobile app, the [Verida Wallet](#). Their ecosystem consists of a range of products and protocols interacting with each other, and openly invites developers and builders to contribute and add to their personalised data innovation. Users can browse encrypted data and apply a set of delegated permissions when consensually sharing it with other users and services. There is a single ERC20 smart contract for Verida Token, Verida have also implemented [Magna](#), a token management and distribution platform for the token lockup requirements.

Token Name: Verida Token (VDA)

Contract Address: N/A

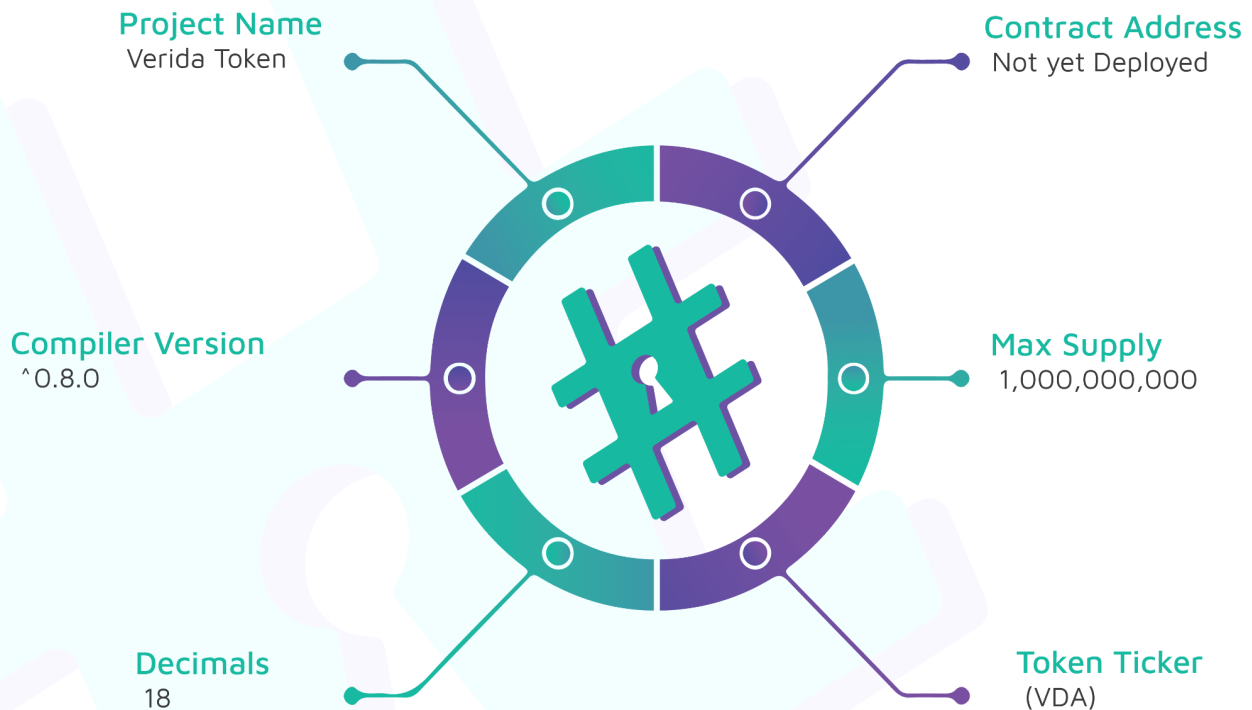
Compiler Version: ^0.8.0

Max Supply: 1,000,000,000

Decimals: 18

Logo:

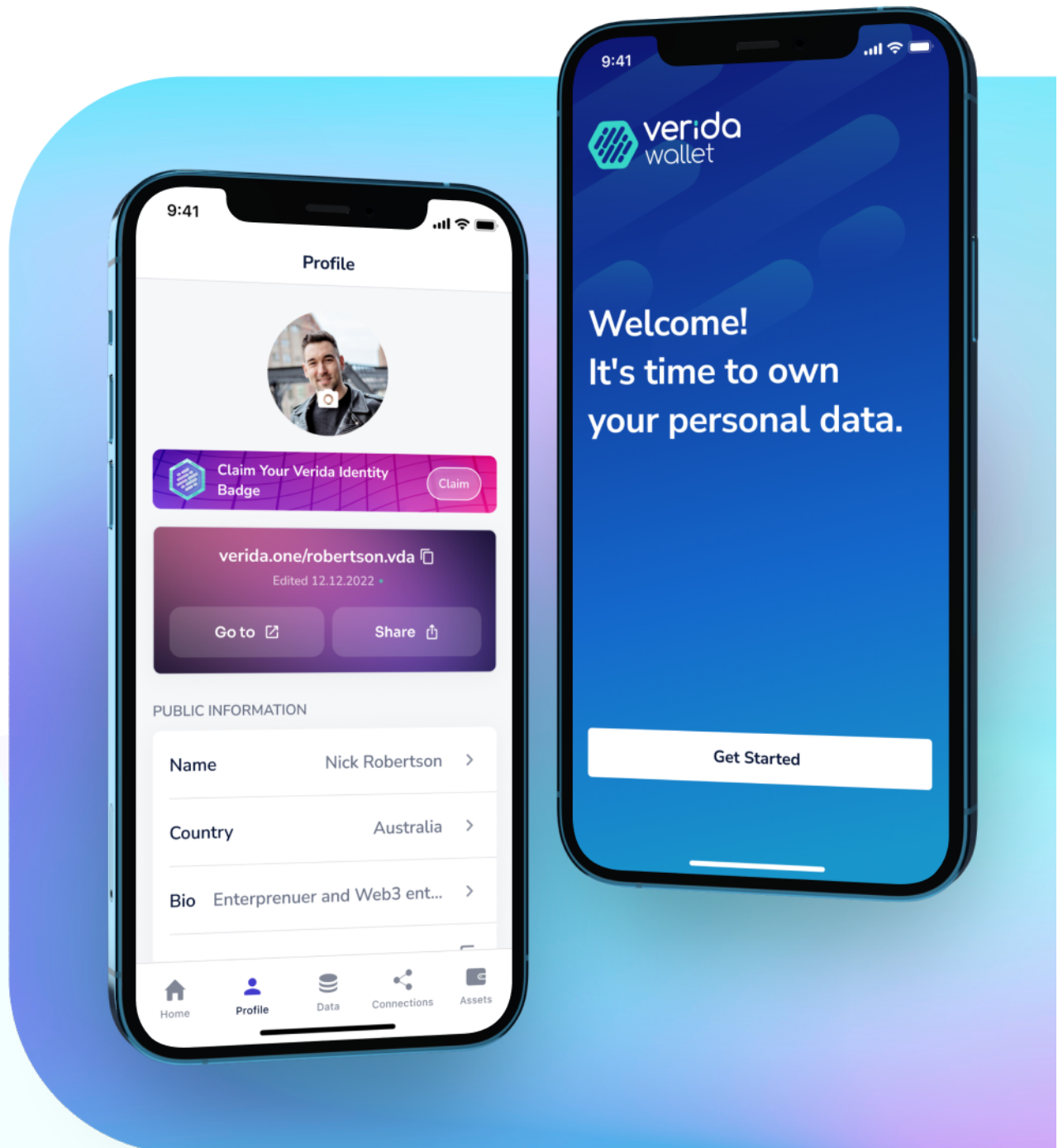
verida network

Visualised Context:

#Hashlock.

Hashlock Pty Ltd

Platform Visualisation:



Verida's Executive Team Leadership

Title	Name	Photo	Links
CEO Co-Founder	Chris Were		Linkedin
COO Co-Founder	Ryan Kris		Linkedin

See their wider team [here](#).

Audit scope

We at Hashlock audited the solidity code within the Verida Token, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken through both manual and software assisted analysis.

Description	Project Review and Security Analysis Report for Verida Protocol Smart Contracts and other factors.
Platform	Ethereum / Solidity
Audit Date	January 24th, 2023
Contract 1	VDA-V1.sol
Contract 1 MD5 Hash	A8F93343290C57E4B3F19E6E086FC215
Contract 2	VDA-Lock.sol
Contract 2 MD5 Hash	FD2B18FBF5EB6334CD2C7E46CBA74EAE
Contract 3	RecipientLockUp.sol - now using Magna
Contract 3 MD5 Hash	00334A79C7EAE33FB35A92F694869DD7
Contract 4	IVDA-Lock.sol
Contract 4 MD5 Hash	7b968b7542489f6f5ff28f95cc8e68cc
Contract 5	IVDA.sol
Contract 5 MD5 Hash	38af004c8dbe9dd73e408113ae3dd9e5
Contract 6	Mock-VDA-V2.sol
Contract 6 MD5 Hash	97cb70905c1d9871b72077d6dc4ad4dc
Contract 7	IRecipientLockUp.sol - now using Magna
Contract 7 MD5 Hash	947475572c937aefe696b45526a972a1

The Verida network contains a range of other smart contracts that will be audited in a subsequent report.

Security Rating

After our audit and analysis, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified multiple high and medium severity vulnerabilities that will need to be addressed before mainnet launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for fully decentralised, highly secure projects with little owner influence.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

0 Critical severity vulnerabilities

2 High severity vulnerabilities

4 Medium severity vulnerabilities

1 Low severity vulnerabilities

6 Very low, informational, best practice severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Reviewed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Reviewed
	Lack of check on input parameters	Passed
	Function input parameters check bypass	Passed
	Correct Access control	Reviewed
	Built in emergency features	Passed
	Correct event logs	Passed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Reviewed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: FAILED

Revised Audit Result: PASSED

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Contract 1 VDA-V1.sol - Name: Verida - Symbol: VDA - Decimal: 18 - Maximum Supply: 1 Billion Fees Specifications: Max Amount (or rate) per wallet: 20% Max Transfer Amount (or rate): Not applicable Max Sell Amount (or rate): 0.1% Other Specifications: Inflation: No fixed inflation, tokens will be granted as “rewards” in a future smart contract. Minting: The contract owner can call the mint function to increase the supply. Token release: Has a hard coded start date. Transfer fees: Not required. Referrals: Not required. Ownership Control: Owner can add lockup types. - now using Magna Owners can exclude accounts from the Max Amount per wallet limit.	Contract achieves this functionality.
Contract 2 VDALock.sol - Owner can add a new lock type. - Owner can add a new lock holder.	Contract achieves this functionality.

- Owner can remove the lock holder address.	
Contract 3 RecipientLockUp.sol Ownership Control: - now using Magna - Owner can add a new lock type. - Owner can withdraw unlocked tokens. - Owner can withdraw unlocked tokens to address. - Owner can add a new recipient address. - Owner can remove the recipient address.	Contract achieves this functionality.

Function List

VeridaToken.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initialising	No Issue
3	__Ownable_init_unchained	internal	access only Initialising	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue
7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	__ERC20Pausable_init	internal	access only Initialising	No Issue
11	__ERC20Pausable_init_unchained	internal	access only Initialising	No Issue
12	_beforeTokenTransfer	internal	Passed	No Issue
13	__AccessControlEnumerable_init	internal	access only Initialising	No Issue
14	__AccessControlEnumerable_init_unchained	internal	access only Initialising	No Issue
15	supportsInterface	read	Passed	No Issue
16	getRoleMember	read	Passed	No Issue
17	getRoleMemberCount	read	Passed	No Issue
18	_grantRole	internal	Passed	No Issue
19	_revokeRole	internal	Passed	No Issue

20	validMint	modifier	Passed	No Issue
21	initialize	external	Passed	No Issue
22	_initSwap	internal	Unused Internal function	Refer Audit Findings
23	decimals	read	Passed	No Issue
24	getTokenPublishTime	external	Passed	No Issue
25	mint	external	access only valid Mint	No Issue
26	burn	external	Passed	No Issue
27	addMinter	external	access only Owner	No Issue
28	revokeMinter	external	access only Owner	No Issue
29	getMinterCount	external	Passed	No Issue
30	getMinterList	external	Passed	No Issue
31	getVersion	external	Passed	No Issue
32	_transfer	internal	Passed	No Issue
33	updateUniswapV2Router	external	Unused Internal function	Refer Audit Findings
34	setAutomatedMarketMakerPair	external	access only Owner	No Issue
35	_setAutomatedMarketMakerPair	write	Passed	No Issue
36	updateMaxAmountPerWalletRate	external	access only Owner	No Issue
37	_updateMaxAmountPerWallet	write	Passed	No Issue
38	updateMaxAmountPerSellRate	external	access only Owner	No Issue

39	_updateMaxAmountPerSell	write	Passed	No Issue
40	excludeFromSellAmountLimit	external	access only Owner	No Issue
41	excludeFromWalletAmountLimit	external	access only Owner	No Issue
42	enableMaxAmountPerSell	external	Anyone can call this function	Refer Audit Findings
43	enableMaxAmountPerWallet	external	Anyone can call this function	Refer Audit Findings

VDA Lock.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initialising	No Issue
3	__Ownable_init_unchained	internal	access only Initialising	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue
7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	__ReentrancyGuard_init	internal	access only Initialising	No Issue
11	__ReentrancyGuard_init_unchained	internal	access only Initialising	No Issue
12	nonReentrant	modifier	Passed	No Issue

13	_nonReentrantBefore	write	Passed	No Issue
14	_nonReentrantAfter	write	Passed	No Issue
15	_reentrancyGuardEntered	internal	Passed	No Issue
16	initialize	external	access only Initialising	No Issue
17	_initLockupType	internal	Passed	No Issue
18	addLockType	external	access only Owner	No Issue
19	_addLockType	write	Passed	No Issue
20	addLockHolder	external	access only Owner	Refer Audit Findings
21	removeLockHolder	external	access only Owner	Refer Audit Findings
22	lockedAmount	external	Passed	No Issue
23	claimableAmount	external	Passed	No Issue
24	_claimableAmount	read	Passed	No Issue
25	claim	external	Transferred 0 amount	Refer Audit Findings

RecipientLockUp.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initializing	No Issue
3	__Ownable_init_unchained	internal	access only Initializing	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue

7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	initialize	write	access only initializer	No Issue
11	_initLockupType	internal	Passed	No Issue
12	addLockType	external	access only Owner	No Issue
13	_addLockType	write	Passed	No Issue
14	withdrawUnlockedTokens	write	access only Owner	No Issue
15	withdrawUnlockedTokensTo	write	access only Owner	No Issue
16	addRecipient	external	access only Owner	No Issue
17	addRecipientWithLockType	external	access only Owner	No Issue
18	_addRecipientWithLockType	write	Passed	No Issue
19	removeRecipient	external	access only Owner	No Issue
20	lockedAmount	external	Passed	No Issue
21	claimableAmount	external	Passed	No Issue
22	_claimableAmount	read	Passed	No Issue
23	claim	external	Transferred 0 amount	Refer Audit Findings

VDA-Lock.sol Function Analysis

A fundamental focus of the audit is Verida's lock contracts, and the token they use. The VDA-Lock.sol and the RecipientLockUp.sol share similar architecture, so our team has decided to include a function analysis section using the VDA-Lock contract to detail its usage. Below are comments on its main functions.

Verida has now implemented [Magna](#), a token management and distribution platform for the token lockup requirements that will replace the need for the lock contract functions.

Function `_initLockUpType`: - now using Magna

```
VDA-Lock.sol

44     function _initLockupType() internal {
45
46         // Investors
47         _addLockType(
48             2 * 365 days,
49             30 days,
50             0,
51             true
52         );
53
54         // Founders, Mozzler grant
55         _addLockType(
56             4 * 365 days,
57             30 days,
58             0,
59             true
60         );
61
62         // Team members
63         _addLockType(
64             4 * 365 days,
65             30 days,
66             365 days,
67             false
68         );
69
70         // Advisors
71         _addLockType(
72             2 * 365 days,
73             30 days,
74             0,
75             true
76         );
77
78         // Community
79         _addLockType(
80             5 * 365 days,
81             30 days,
82             0,
83             true
84         );
85     }
```

Initialising the contract calls the internal function that creates lock types for each stakeholder type using different periods.

Function `addLockType`: - now using Magna

```

90     function addLockType(uint256 lockDuration, uint256 releaseInterval, uint256 releaseDelay, bool isValidFromTGE) external onlyOwner override {
91         require(lockDuration > 0, "Invalid lock duration");
92         require(releaseInterval > 0, "Invalid release interval");
93         _addLockType(lockDuration, releaseInterval, releaseDelay, isValidFromTGE);
94     }
95
96     function _addLockType(uint256 lockDuration, uint256 releaseInterval, uint256 releaseDelay, bool isValidFromTGE) private {
97         lockTypeCount++;
98         lockTypeInfo[lockTypeCount] = LockTypeInfo(
99             lockDuration,
100             releaseInterval,
101             releaseDelay,
102             isValidFromTGE
103         );
104
105         emit AddLockType(lockTypeCount, lockDuration, releaseInterval, releaseDelay, isValidFromTGE);
106     }

```

The owner can add lock types, and remove them in a corresponding set of functions.

Function `addLockHolder` and `removeLockHolder`: - now using Magna

```

111     function addLockHolder(address to, uint8 _lockType, uint256 _lockAmount, uint256 _lockStart) external onlyOwner nonReentrant override {
112         require(to != address(0x0), "Invalid zero address");
113         require(_lockType > 0 && _lockType <= lockTypeCount, "Invalid lock type");
114         require(_lockAmount > 0, "Invalid lock amount");
115
116         uint256 tokenPublishTime = token.getTokenPublishTime();
117         if (lockTypeInfo[_lockType].isValidFromTGE) {
118             require(block.timestamp < tokenPublishTime, "Token published");
119         } else {
120             require(_lockStart >= block.timestamp, "Invalid lock start time");
121         }
122
123         UserLockInfo storage userInfo = userLockInfo[to];
124         if (userInfo.lockType != 0) {
125             token.burn(address(this), userInfo.lockAmount);
126         }
127
128         token.mint(address(this), _lockAmount);
129
130         userInfo.lockType = _lockType;
131         userInfo.lockAmount = _lockAmount;
132         userInfo.lockStart = lockTypeInfo[_lockType].isValidFromTGE ?
133             ( tokenPublishTime ) :
134             (_lockStart );
135         userInfo.released = 0;
136
137         emit AddLockHolder(to, _lockType, _lockAmount);
138     }

```

```

VDA-Lock.sol

143     function removeLockHolder(address to) external onlyOwner nonReentrant override {
144         uint256 tokenPublishTime = token.getTokenPublishTime();
145
146         UserLockInfo storage userInfo = userLockInfo[to];
147         require(userInfo.lockType > 0 && userInfo.lockType <= lockTypeCount, "Not a lock holder");
148
149         LockTypeInfo storage lockInfo = lockTypeInfo[userInfo.lockType];
150
151         if (lockInfo.isValidFromTGE) {
152             require(block.timestamp < tokenPublishTime, "Token published");
153         }
154
155         token.burn(address(this), userLockInfo[to].lockAmount - userLockInfo[to].released);
156         delete userLockInfo[to];
157
158         emit RemoveLockHolder(to);
159     }

```

The owner can add and remove lock holder addresses. These functions have several suggested improvements in the audit findings section, mainly revolving around the potential for lock holders to have their tokens burnt.

Function claim: - now using Magna

```

VDA-Lock.sol

198     function claim() external nonReentrant override {
199         uint256 amount = _claimableAmount(msg.sender);
200
201         if (amount > 0) {
202             UserLockInfo storage userInfo = userLockInfo[msg.sender];
203
204             userInfo.released += amount;
205             IERC20Upgradeable((address(token))).safeTransfer(msg.sender, amount);
206
207             emit Claim(msg.sender, amount);
208         }
209     }
210 }

```

Updating the amount that the address has released, which is effectively its balance, before transferring tokens to the function caller.

Code Quality

This audit scope involves the solidity smart contracts of the Verida project, as outlined in the Audit Scope section. All contracts, libraries and interfaces follow standard best practices and avoid unnecessary complexity that increases the likelihood of exploitation.

All code is well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Verida Protocol's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are very helpful in understanding the overall architecture of the protocol.

We were also provided with project documentation and links to assist with our understanding of the project's architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
Critical	Critical vulnerabilities are of immediate concern, and allow hackers or malicious code to steal funds, take ownership of the contract, or other devastating actions.
High	High vulnerabilities are not attacked as easily, but still have significant implications on the protocol.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Informational	These are mostly preferred factors that won't affect security.

Audit Findings

Critical

No critical vulnerabilities were found.

High

- 1) **VDA-V1.sol**: Anyone can execute the 'enableMaxAmountPerWallet' and 'enableMaxAmountPerSell' functions.

```
VDA-V1.sol

317  /**
318   * @dev enable/disable MaxAmountPerSell
319   */
320  function enableMaxAmountPerSell(bool isEnabled) external {
321      require(isMaxAmountPerSellEnabled != isEnabled);
322      isMaxAmountPerSellEnabled = isEnabled;
323      emit EnableMaxAmountPerSell(isEnabled);
324  }
325
326  /**
327   * @dev enable/disable MaxAmountPerWallet
328   */
329  function enableMaxAmountPerWallet(bool isEnabled) external {
330      require(isMaxAmountPerWalletEnabled != isEnabled);
331      isMaxAmountPerWalletEnabled = isEnabled;
332      emit EnableMaxAmountPerWallet(isEnabled);
333  }
334 }
```

"enableMaxAmountPerWallet" and "enableMaxAmountPerSell" functions are accessible to anyone. So any user can execute these functions and manipulate the transfer of the tokens.

Resolution: We suggest adding the onlyOwner modifier to these functions.

Resolved

- 2) **VDA-Lock.sol and RecipientLockUp.sol:** Lock holders funds can be lost if the addLockHolder function is called by the owner for an existing lock holder address.

```

VDA-Lock.sol

123      UserLockInfo storage userInfo = userLockInfo[to];
124      if (userInfo.lockType != 0) {
125          token.burn(address(this), userInfo.lockAmount);
126      }
127
128      token.mint(address(this), _lockAmount);

```

If the owner address accidentally or otherwise calls the addLockHolder function for an existing lock holder address, all of their locked tokens are burnt.

Resolution: We suggest simply having:

```
require(userInfo.lockType == 0, "Can not add an existing lock holder")
```

at the beginning of the function and to remove the if statement on line 123.

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

Medium

- 1) **VDA-Lock.sol:** Removing a lock holder address burns its tokens.

```

VDA-Lock.sol

155      token.burn(address(this), userLockInfo[to].lockAmount - userLockInfo[to].released);
156      delete userLockInfo[to];

```

Currently removing a lock holder burns its tokens. This may be desired, if so, please ignore.

Resolution:

Transfer the address its locked tokens before removing it as a lock holder, whilst ensuring that the tokens are ready to be released, as per your desired specifications.

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

- 2) **Mock-VDA-V2.sol** does not integrate completely with VDA-Lock.sol as currently constructed.

Screen shot from VDA-Lock.sol

```

111     function addLockHolder(address to, uint8 _lockType, uint256 _lockAmount, uint256 _lockStart) external onlyOwner nonReentrant override {
112         require(to != address(0x0), 'Invalid zero address');
113         require(_lockType > 0 && _lockType <= lockTypeCount, "Invalid lock type");
114         require(_lockAmount > 0, "Invalid lock amount");
115
116         uint256 tokenPublishTime = token.getTokenPublishTime();

```

VDA-Lock.sol needs to call token.getTokenPublishTime, however the mock V2 does not have this function, and thus you couldn't add lock holders.

Resolution: Add the getTokenPublishTime to the Mock-VDA-V2.sol contract if it is intended to have complete functionality.

Acknowledged: Mock token is not relevant and will not be utilised.

- 3) **VDA-V1.sol:** Publish time is already set to the past.

```

95         tokenPublishTime = 1672531200; //2023-1-1 0:0:0 UTC

```

Unless this is the desired spec, the tokenPublishTime variable will need to be set to the actual publish time.

Resolution: Update the token publish time to the actual publish time.

Resolved: Removed as there is no longer a lock contract to call getTokenPublishTime. Will need to be re-added if Magna.so specs require it.

- 4) **VDA-V1.sol:** Internal function to initialise the uniswap features is not called anywhere.

```
VDA-V1.sol

111     function _initSwap() internal {
112         address routerAddress =
113             0x1Ed675D5e63314B760162A3D1Cae1803DCFC87C7 // BSC TestNet (ME)
114             // 0x9Ac64Cc6e4415144C455BD8E4837Fea55603e5c3 //TestNet (John)
115             // 0x10ED43C718714eb63d5aA57B78B54704E256024E // BSC Mainnet
116     ;
```

It seems this should be called from the initializer function, or at least somewhere else in the contract to make it accessible. Currently, the uniswap features could not be initialised.

Resolution: call _initSwap from the initializer function or at least anywhere else in the contract if this function needs to be used, otherwise, remove unused code.

Resolved: Code no longer needed, removed in new spec.

Low

- 1) **VDA-Lock.sol:** maximum lock types before DOS.

```
VDA-Lock.sol

18     /** @dev number of lock types registered */
19     uint8 public lockTypeCount;
20
21     /**
22      * @dev LockTypeInfo of each LockType
23      * LockType => LockTypeInfo
24      */
25     mapping(uint8 => LockTypeInfo) public lockTypeInfo;
```

There is a maximum of 256 lock types that can be held due to the uint8. These can only be added via the owner, and it is very unlikely that there will ever be a need for more than 256.

Resolution: Increase the amount of storage used if you think this may be an issue depending on your future usage of the contract.

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

Very Low / Informational / Best practices:

- 1) **VDA-Lock.sol:** Claim function does not truly require nonReentrant modifier.

```

198     function claim() external nonReentrant override {
199         uint256 amount = _claimableAmount(msg.sender);
200
201         if (amount > 0) {
202             UserLockInfo storage userInfo = userLockInfo[msg.sender];
203
204             userInfo.released += amount;
205             IERC20Upgradeable((address(token))).safeTransfer(msg.sender, amount);
206
207             emit Claim(msg.sender, amount);
208         }
209     }
210 }

```

As balance is updated on line 204 before the transfer, and the transfer uses SafeTransfer, reentrancy is impossible as the function follows the correct check, effect, interact order. The modifier is fine to have, however it is a slight waste of gas.

Resolution: Remove nonReentrant modifier if you wish to save gas.

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

- 2) **VDA-Lock.sol:** addLockHolder does not follow the correct check, effect, interact order.

```

VDA-Lock.sol
111     function addLockHolder(address to, uint8 _lockType, uint256 _lockAmount, uint256 _lockStart) external onlyOwner nonReentrant override {
112         require(to != address(0x0), 'Invalid zero address');
113         require(_lockType > 0 && _lockType <= lockTypeCount, "Invalid lock type");
114         require(_lockAmount > 0, "Invalid lock amount");
115
116         uint256 tokenPublishTime = token.getTokenPublishTime();
117         if (lockTypeInfo[_lockType].isValidFromTGE) {
118             require(block.timestamp < tokenPublishTime, "Token published");
119         } else {
120             require(_lockStart >= block.timestamp, "Invalid lock start time");
121         }
122
123         UserLockInfo storage userInfo = userLockInfo[to];
124         if (userInfo.lockType != 0) {
125             token.burn(address(this), userInfo.lockAmount);
126         }
127
128         token.mint(address(this), _lockAmount);
129
130         userInfo.lockType = _lockType;
131         userInfo.lockAmount = _lockAmount;
132         userInfo.lockStart = lockTypeInfo[_lockType].isValidFromTGE ?
133             ( tokenPublishTime ) :
134             ( _lockStart );
135         userInfo.released = 0;
136
137         emit AddLockHolder(to, _lockType, _lockAmount);
138     }

```

userInfo information is updated after the token.mint call, whereas the mint should really happen after, as a best practice way to ensure that info is always updated in the event of the mint.

Resolution: Update to best practice ordering, with token.mint call after the userInfo changes.

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

- 3) **VDA-Lock.sol, VDA-V1.sol and RecipientLockUp.sol:** Owners can be set to address 0 without any checks.

Owners can be set to address 0, which may be a future decentralisation method, however there is a low chance owners can accidentally renounce ownership and lose control of the function.

Resolution: Either add a check that address is not 0, or a multi step process to renounce ownership when address 0 is called.

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

- 4) **Verida Protocol:** Pragma Solidity Version

Using solidity ^0.8.17 is current best practice and may prevent compiler bugs.

Resolution: Update contracts to pragma solidity ^0.8.17

Acknowledged

5) **VDA-V1.sol**: Duplicate decimal variable.

VDA-V1.sol

```
23      uint8 public constant DECIMAL = 18;
```

VDA-V1.sol

```
130      /** @dev Decimals of Verida token */  
131      function decimals() public view virtual override returns (uint8) {  
132          return DECIMAL;  
133      }
```

“DECIMAL” has the same function as “decimals” in the ERC20 standard. 18 decimals is set by default. We suggest removing this variable and the decimals() function in the code.

See ERC20.sol standard: VDA-V1 already inherits this via ERC20PausableUpgradable.sol.

```

VDA-V1.sol

472     /**
473      * @dev Returns the number of decimals used to get its user representation.
474      * For example, if `decimals` equals `2`, a balance of `505` tokens should
475      * be displayed to a user as `5.05` ( $505 / 10 ** 2$ ).
476      *
477      * Tokens usually opt for a value of 18, imitating the relationship between
478      * Ether and Wei. This is the value {ERC20} uses, unless this function is
479      * overridden;
480      *
481      * NOTE: This information is only used for _display_ purposes: it in
482      * no way affects any of the arithmetic of the contract, including
483      * {IERC20-balanceOf} and {IERC20-transfer}.
484      */
485     function decimals() public view virtual override returns (uint8) {
486         return 18;
487     }

```

Resolution: Remove the DECIMAL variable and its corresponding function.

Acknowledged

6) **VDA-Lock.sol, RecipientLockUp.sol:** Gas wasted on claim function.

```

VDA-Lock.sol

198     function claim() external nonReentrant override {
199         uint256 amount = _claimableAmount(msg.sender);
200
201         if (amount > 0) {
202             UserLockInfo storage userInfo = userLockInfo[msg.sender];
203
204             userInfo.released += amount;
205             IERC20Upgradeable((address(token))).safeTransfer(msg.sender, amount);
206
207             emit Claim(msg.sender, amount);
208         }
209     }
210 }

```

The if statement does not allow 0 amount values to be claimed. This if statement costs gas and increases complexity unnecessarily.

Resolution: We suggest using a require statement after the amount local variable is determined, to check:

```
require(amount != 0, "No claimable tokens")
```

And then take the transfer and other logic out of the if statement and remove the if statement.

Additionally, the uint for 'amount' does not need to be specified to be 256, as that is the default uint value.

Example revised claim function (*override not written in example but should still be included*):

```
revised_claim_function.sol

1  function claim() external nonReentrant override {
2
3      uint amount = _claimableAmount(msg.sender);
4
5      require(amount != 0, "No claimable tokens");
6
7      UserLockInfo storage userInfo = userLockInfo[msg.sender];
8
9      userInfo.released += amount;
10     IERC20Upgradeable((address(token))).safeTransfer(msg.sender, amount);
11
12     emit Claim(msg.sender, amount);
13 }
```

Resolved: Relevant contracts removed as Verida will now use Magna.so contracts, externally audited elsewhere.

Centralisation

The Verida protocol values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on private key security, and internal team responsibility.



Centralised

Decentralised

Owner executable functions

Verida has now implemented [Magna](#), a token management and distribution platform for the token lockup requirements that will replace the need for the lock contract functions.

RecipientLockUp.sol - now using Magna

- addLockType: Owner can add a new lock type.
- withdrawUnlockedTokens: Owner can withdraw unlocked tokens.
- withdrawUnlockedTokensTo: Owner can withdraw unlocked tokens to address.
- addRecipient: Owner can add a new recipient address.
- addRecipientWithLockType: Owner can add a new recipient address with lock type.
- removeRecipient: Owner can remove the recipient address.

VDALock.sol - now using Magna

- addLockType: Owner can add a new lock type.
- addLockHolder: Owner can add a new lock holder address.
- removeLockHolder: Owner can remove the lock holder address.

VeridaToken.sol

- addMinter: Owner can add a new minter address.
- revokeMinter: Owner can revoke minter address.
- updateUniswapV2Router: Owner can update uniswapV2Router to a new address.
- setAutomatedMarketMakerPair: Owner can set enable/disable automated market maker pair address.
- updateMaxAmountPerWalletRate: Owner can update maximum amount per wallet rate.
- updateMaxAmountPerSellRate: Owner can update maximum amount per sell percent.
- excludeFromSellAmountLimit: Owner can exclude account from sell amount limit.
- excludeFromWalletAmountLimit: Owner can exclude account from wallet amount limit.

Conclusion

After Hashlocks analysis, the Verida protocol seems to have a sound and well tested code base, pending the resolution of our findings. Overall, most of the code is correctly ordered and follows industry best practices.

The project does sacrifice decentralisation for security, as there is a large amount of access control, as well as the manual release of rewards. Hashlock believes this is a sound decision.

As our findings have now been resolved, Hashlock believes Verida's solidity code base is ready for deployment.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

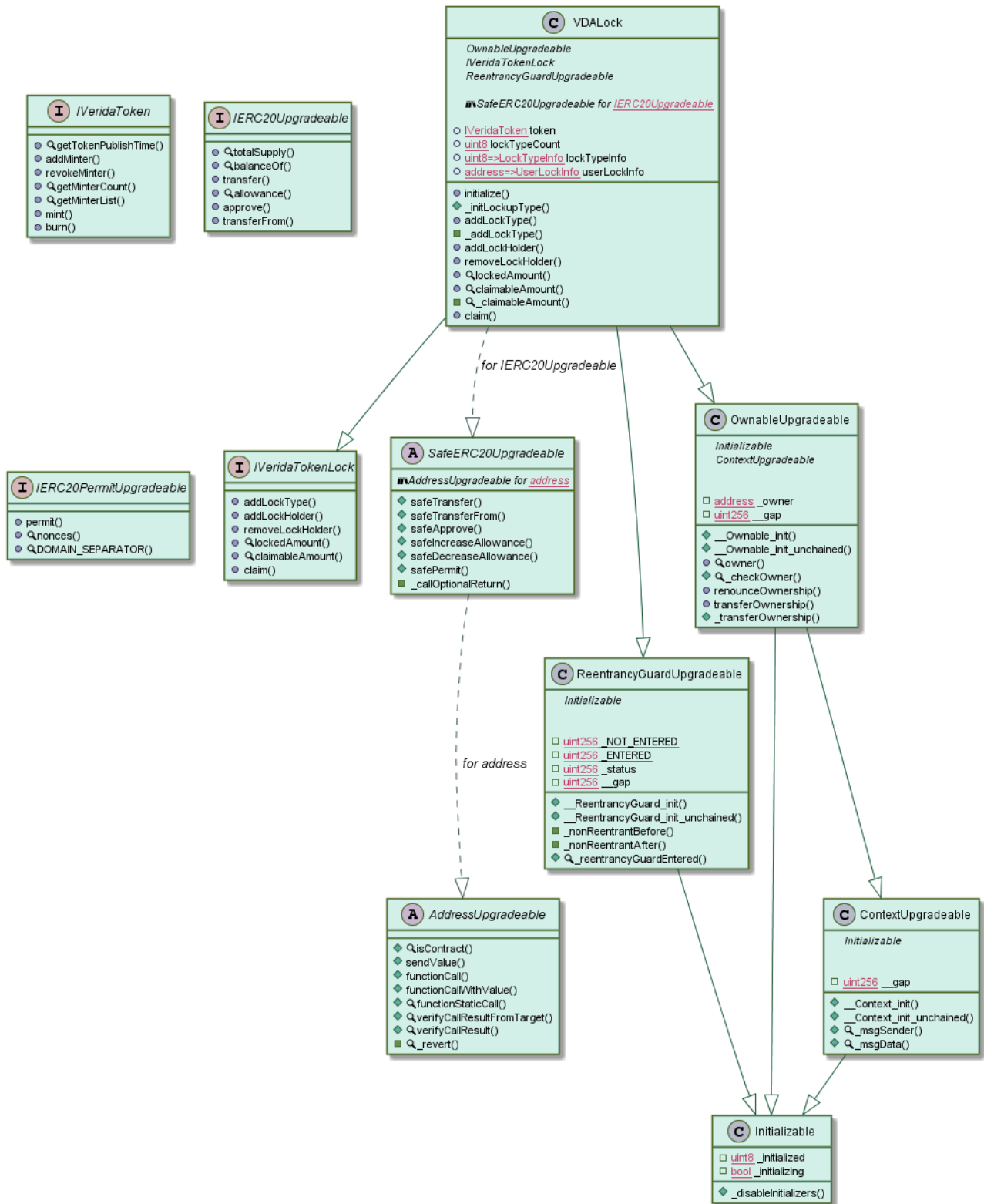
Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

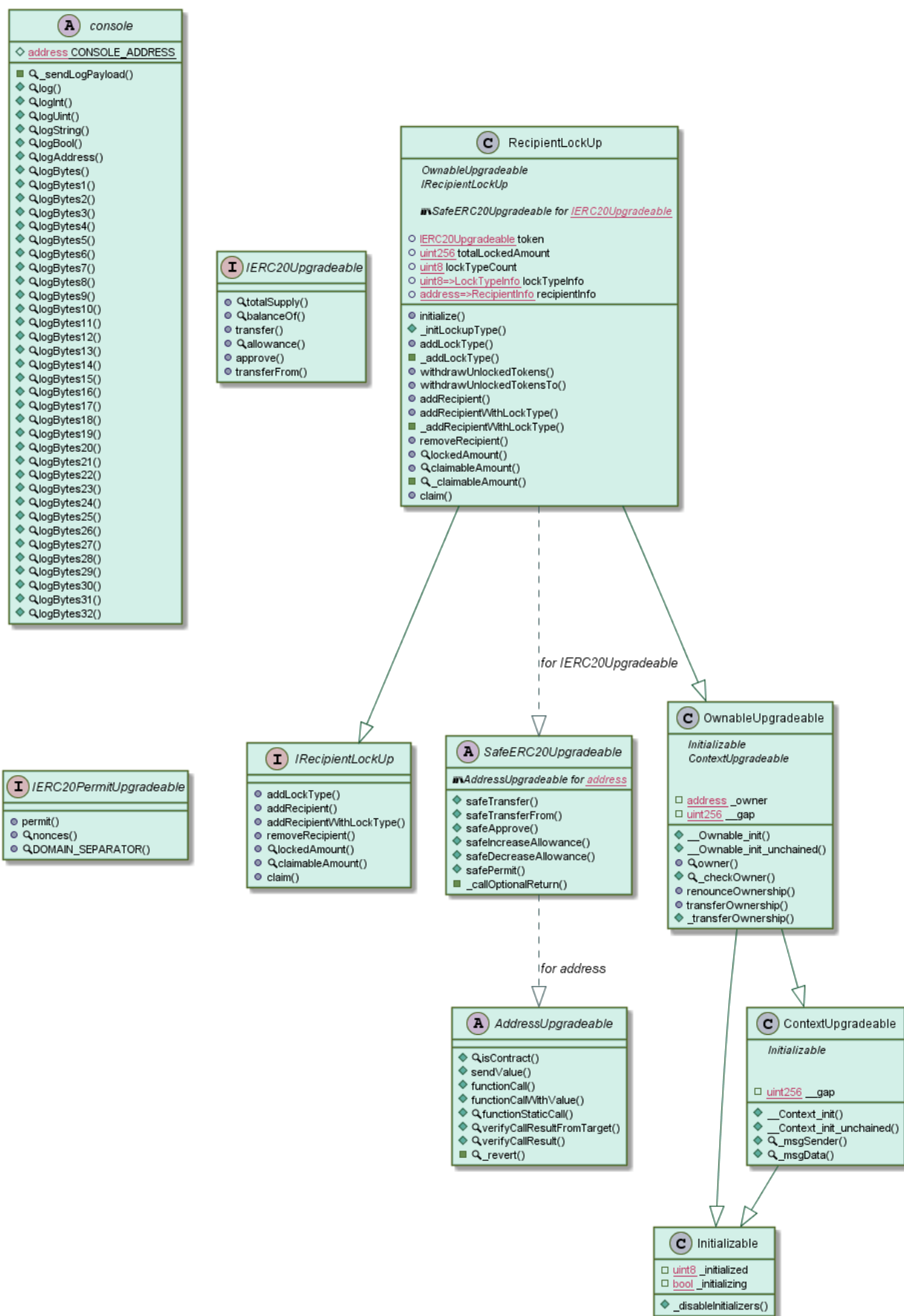
Contact: info@hashlock.com.au



VDA Lock Diagram - now using Magna



RecipientLockUp Diagram - now using Magna



Slither Results Log

Slither Log >> VeridaToken.sol

```

Reentrancy in VeridaToken._initSwap() (VeridaToken.sol#1742-1759):
  External calls:
    - uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory()).createPair(address(this),uniswapV2Router.WETH()) (VeridaToken.sol#1753-1756)
  State variables written after the call(s):
    - _setAutomatedMarketMakerPair(uniswapV2Pair,true) (VeridaToken.sol#1758)
      - automatedMarketMakerPairs[pair] = value (VeridaToken.sol#1883)
  Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2

Reentrancy in VeridaToken._initSwap() (VeridaToken.sol#1742-1759):
  External calls:
    - uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory()).createPair(address(this),uniswapV2Router.WETH()) (VeridaToken.sol#1753-1756)
  Event emitted after the call(s):
    - SetAutomatedMarketMakerPair(pair,value) (VeridaToken.sol#1885)
      - _setAutomatedMarketMakerPair(uniswapV2Pair,true) (VeridaToken.sol#1758)
  Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

Pragma version^0.8.0 (VeridaToken.sol#2) allows old versions
solc-0.8.0 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (VeridaToken.sol#260-265):
  - (success) = recipient.call{value: amount}() (VeridaToken.sol#263)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (VeridaToken.sol#287-296):
  - (success,returndata) = target.call{value: value}(data) (VeridaToken.sol#294)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (VeridaToken.sol#302-309):
  - (success,returndata) = target.staticcall(data) (VeridaToken.sol#307)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Function IUniswapV2Router01.WETH() (VeridaToken.sol#6) is not in mixedCase
Function IUniswapV2Pair.DOMAIN_SEPARATOR() (VeridaToken.sol#170) is not in mixedCase
Function IUniswapV2Pair.PERMIT_TYPEHASH() (VeridaToken.sol#171) is not in mixedCase
Function IUniswapV2Pair.MINIMUM_LIQUIDITY() (VeridaToken.sol#188) is not in mixedCase
Function ContextUpgradeable.__Context_init() (VeridaToken.sol#430-431) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (VeridaToken.sol#433-434) is not in mixedCase
Variable ContextUpgradeable.__gap (VeridaToken.sol#443) is not in mixedCase
Function ERC20Upgradeable.__ERC20_init(string,string) (VeridaToken.sol#457-459) is not in mixedCase
Function ERC20Upgradeable.__ERC20_init_unchained(string,string) (VeridaToken.sol#461-464) is not in mixedCase
Variable ERC20Upgradeable.__gap (VeridaToken.sol#621) is not in mixedCase
Function PausableUpgradeable.__Pausable_init() (VeridaToken.sol#640-642) is not in mixedCase
Function PausableUpgradeable.__Pausable_init_unchained() (VeridaToken.sol#644-646) is not in mixedCase
Variable PausableUpgradeable.__gap (VeridaToken.sol#722) is not in mixedCase
Function ERC20PausableUpgradeable.__ERC20Pausable_init() (VeridaToken.sol#727-729) is not in mixedCase
Function ERC20PausableUpgradeable.__ERC20Pausable_init_unchained() (VeridaToken.sol#731-732) is not in mixedCase
Variable ERC20PausableUpgradeable.__gap (VeridaToken.sol#755) is not in mixedCase
Function ERC165Upgradeable.__ERC165_init() (VeridaToken.sol#1274-1275) is not in mixedCase
Function ERC165Upgradeable.__ERC165_init_unchained() (VeridaToken.sol#1277-1278) is not in mixedCase
Variable ERC165Upgradeable.__gap (VeridaToken.sol#1291) is not in mixedCase
Function AccessControlUpgradeable.__AccessControl_init() (VeridaToken.sol#1295-1296) is not in mixedCase
Function AccessControlUpgradeable.__AccessControl_init_unchained() (VeridaToken.sol#1298-1299) is not in mixedCase
Variable AccessControlUpgradeable.__gap (VeridaToken.sol#1503) is not in mixedCase
Function AccessControlEnumerableUpgradeable.__AccessControlEnumerable_init() (VeridaToken.sol#1507-1508) is not in mixedCase
Function AccessControlEnumerableUpgradeable.__AccessControlEnumerable_init_unchained() (VeridaToken.sol#1510-1511) is not in mixedCase
Variable AccessControlEnumerableUpgradeable.__gap (VeridaToken.sol#1568) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (VeridaToken.sol#1580-1582) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (VeridaToken.sol#1584-1586) is not in mixedCase
Variable OwnableUpgradeable.__gap (VeridaToken.sol#1645) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions

Variable IUniswapV2Router01.addLiquidity(address,address,uint256,uint256,uint256,uint256,address,uint256).amountADesired (VeridaToken.sol#11) is too similar to IUniswapV2Router01.addLiquidity(address,address,uint256,uint256,uint256,uint256,address,uint256).amountBDesired (VeridaToken.sol#12)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#variable-names-too-similar
VeridaToken.sol analyzed (23 contracts with 84 detectors), 86 result(s) found

```

Slither Log >> VDALock.sol - now using Magna

```

VDALock.addLockHolder(address,uint8,uint256,uint256) (VDALock.sol#648-675) uses timestamp for comparisons
  Dangerous comparisons:
    - require(bool,string)(block.timestamp < tokenPublishTime,Token published) (VDALock.sol#655)
    - require(bool,string)(lockStart >= block.timestamp,Invalid lock start time) (VDALock.sol#657)
VDALock.removeLockHolder(address) (VDALock.sol#680-696) uses timestamp for comparisons
  Dangerous comparisons:
    - require(bool,string)(block.timestamp < tokenPublishTime,Token published) (VDALock.sol#689)
VDALock.claimableAmount(address) (VDALock.sol#713-730) uses timestamp for comparisons
  Dangerous comparisons:
    - userInfo.lockType == 0 || block.timestamp < (userInfo.lockStart + lockInfo.releaseInterval + lockInfo.releaseDelay) (VDALock.sol#718)
    - block.timestamp >= (userInfo.lockStart + lockInfo.lockDuration + lockInfo.releaseDelay) (VDALock.sol#722)
VDALock.claim() (VDALock.sol#735-746) uses timestamp for comparisons
  Dangerous comparisons:
    - amount > 0 (VDALock.sol#738)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp

Pragma version^0.8.0 (VDALock.sol#2) allows old versions
solc-0.8.0 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

```

```

Pragma version^0.8.0 (VDALock.sol#2) allows old versions
solc-0.8.0 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (VDALock.sol#134-139):
- (success) = recipient.call{value: amount}() (VDALock.sol#137)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (VDALock.sol#161-170):
- (success,returndata) = target.call{value: value}(data) (VDALock.sol#168)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (VDALock.sol#176-183):
- (success,returndata) = target.staticcall(data) (VDALock.sol#181)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Function IERC20PermitUpgradeable.DOMAIN_SEPARATOR() (VDALock.sol#259) is not in mixedCase
Function ReentrancyGuardUpgradeable.__ReentrancyGuard_init() (VDALock.sol#405-407) is not in mixedCase
Function ReentrancyGuardUpgradeable.__ReentrancyGuard_init_unchained() (VDALock.sol#409-411) is not in mixedCase
Variable ReentrancyGuardUpgradeable.__gap (VDALock.sol#453) is not in mixedCase
Function ContextUpgradeable.__Context_init() (VDALock.sol#457-458) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (VDALock.sol#460-461) is not in mixedCase
Variable ContextUpgradeable.__gap (VDALock.sol#470) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (VDALock.sol#481-483) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (VDALock.sol#485-487) is not in mixedCase
Variable OwnableUpgradeable.__gap (VDALock.sol#546) is not in mixedCase
Parameter VDALock.addLockHolder(address,uint8,uint256,uint256)._lockType (VDALock.sol#648) is not in mixedCase
Parameter VDALock.addLockHolder(address,uint8,uint256,uint256)._lockAmount (VDALock.sol#648) is not in mixedCase
Parameter VDALock.addLockHolder(address,uint8,uint256,uint256)._lockStart (VDALock.sol#648) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
VDALock.sol analyzed (11 contracts with 84 detectors), 42 result(s) found

```

Slither Log >> RecipientLockUp.sol - now using Magna

```

Reentrancy in RecipientLockUp.claim() (RecipientLockUp.sol#2162-2175):
  External calls:
  - IERC20Upgradeable((address(token))).safeTransfer(msg.sender,amount) (RecipientLockUp.sol#2169)
  State variables written after the call(s):
  - totalLockedAmount -= amount (RecipientLockUp.sol#2171)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2

Reentrancy in RecipientLockUp.claim() (RecipientLockUp.sol#2162-2175):
  External calls:
  - IERC20Upgradeable((address(token))).safeTransfer(msg.sender,amount) (RecipientLockUp.sol#2169)
  Event emitted after the call(s):
  - Claim(msg.sender,amount) (RecipientLockUp.sol#2173)
Reentrancy in RecipientLockUp.withdrawUnlockedTokensTo(address) (RecipientLockUp.sol#2060-2067):
  External calls:
  - token.safeTransfer(to,unlockedAmount) (RecipientLockUp.sol#2064)
  Event emitted after the call(s):
  - WithdrawTokens(to,unlockedAmount) (RecipientLockUp.sol#2066)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

RecipientLockUp.withdrawUnlockedTokensTo(address) (RecipientLockUp.sol#2060-2067) uses timestamp for comparisons
  Dangerous comparisons:
  - require(bool,string)(unlockedAmount > 0,No tokens) (RecipientLockUp.sol#2062)
RecipientLockUp._addRecipientWithLockType(address,uint256,uint256,uint8) (RecipientLockUp.sol#2086-2109) uses timestamp for comparisons
  Dangerous comparisons:
  - require(bool,string)(_lockStart >= block.timestamp,Invalid lock start time) (RecipientLockUp.sol#2092)
  - require(bool,string)((totalLockedAmount + _lockAmount) <= tokenAmount,Insufficient token amount) (RecipientLockUp.sol#2100)

Pragma version^0.8.0 (RecipientLockUp.sol#2) allows old versions
solc-0.8.0 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (RecipientLockUp.sol#1631-1636):
- (success) = recipient.call{value: amount}() (RecipientLockUp.sol#1634)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (RecipientLockUp.sol#1658-1667):
- (success,returndata) = target.call{value: value}(data) (RecipientLockUp.sol#1665)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (RecipientLockUp.sol#1673-1680):
- (success,returndata) = target.staticcall(data) (RecipientLockUp.sol#1678)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Contract console (RecipientLockUp.sol#5-1533) is not in CapWords
Function IERC20PermitUpgradeable.DOMAIN_SEPARATOR() (RecipientLockUp.sol#1756) is not in mixedCase
Function ContextUpgradeable.__Context_init() (RecipientLockUp.sol#1886-1887) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (RecipientLockUp.sol#1889-1890) is not in mixedCase
Variable ContextUpgradeable.__gap (RecipientLockUp.sol#1899) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (RecipientLockUp.sol#1910-1912) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (RecipientLockUp.sol#1914-1916) is not in mixedCase
Variable OwnableUpgradeable.__gap (RecipientLockUp.sol#1975) is not in mixedCase
Parameter RecipientLockUp.addRecipient(address,uint256,uint256)._lockAmount (RecipientLockUp.sol#2072) is not in mixedCase
Parameter RecipientLockUp.addRecipient(address,uint256,uint256)._lockStart (RecipientLockUp.sol#2072) is not in mixedCase
Parameter RecipientLockUp.addRecipientWithLockType(address,uint256,uint256,uint8)._lockAmount (RecipientLockUp.sol#2079) is not in mixedCase
Parameter RecipientLockUp.addRecipientWithLockType(address,uint256,uint256,uint8)._lockStart (RecipientLockUp.sol#2079) is not in mixedCase
Parameter RecipientLockUp.addRecipientWithLockType(address,uint256,uint256,uint8)._lockType (RecipientLockUp.sol#2079) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
RecipientLockUp.sol analyzed (10 contracts with 84 detectors), 423 result(s) found

```

Solidity Static Analysis

VeridaToken.sol

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in VeridaToken._initSwap(): Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 111:4:

Gas & Economy

Gas costs:

Gas requirement of function VeridaToken.initialize is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 86:4:

Gas costs:

Gas requirement of function VeridaToken.enableMaxAmountPerWallet is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 329:4:

Miscellaneous

Constant/View/Pure functions:

VeridaToken._transfer(address,address,uint256) : Potentially should be constant/view/pure but is not. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 208:4:

No return:

`IVeridaToken.getMinterList()`: Defines a return type but never explicitly returns a value.

Pos: 29:4:

Guard conditions:

Use `"assert(x)"` if you never ever want `x` to be false, not in any circumstance (apart from a bug in your code). Use `"require(x)"` if `x` can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 330:8:

Data truncated:

Division of integer values yields an integer value again. That means e.g. $10 / 100 = 0$ instead of 0.1 since the result is an integer again. This does not hold for division of (only) literal values since those yield rational constants.

Pos: 296:27:

VDALock.sol - now using Magna

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in `VDALock.addLockHolder(address,uint8,uint256,uint256)`: Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 111:4:

Block timestamp:

Use of `"block.timestamp"`: `"block.timestamp"` can be influenced by miners to a certain degree. That means that a miner can "choose" the `block.timestamp`, to a certain degree, to change the outcome of a transaction in the mined block.

[more](#)

Pos: 190:33:

Gas & Economy

Gas costs:

Gas requirement of function `VDALock.initialize` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 32:4:

Gas costs:

Gas requirement of function `VDALock.claim` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 198:4:

Miscellaneous

Constant/View/Pure functions:

`IVeridaToken.burn(address,uint256)` : Potentially should be constant/view/pure but is not. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 39:4:

Guard conditions:

Use `"assert(x)"` if you never ever want `x` to be false, not in any circumstance (apart from a bug in your code). Use `"require(x)"` if `x` can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 152:12:

Delete from dynamic array:

Using `"delete"` on an array leaves a gap. The length of the array remains the same. If you want to remove the empty position you need to shift items manually and update the `"length"` property.

[more](#)

Pos: 156:8:

Data truncated:

Division of integer values yields an integer value again. That means e.g. $10 / 100 = 0$ instead of 0.1 since the result is an integer again. This does not hold for division of (only) literal values since those yield rational constants.

Pos: 190:32:

RecipientLockUp.sol - now using Magna

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in `SafeERC20Upgradeable.safeApprove(contract IERC20Upgradeable,address,uint256)`: Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 46:4:

Miscellaneous

Constant/View/Pure functions:

`SafeERC20Upgradeable._callOptionalReturn(contract IERC20Upgradeable,bytes)` : Potentially should be constant/view/pure but is not.

[more](#)

Pos: 105:4:

Guard conditions:

Use `"assert(x)"` if you never ever want `x` to be false, not in any circumstance (apart from a bug in your code). Use `"require(x)"` if `x` can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 113:12:

Solhint Linter

VeridaToken.sol

```
VDA-V1.sol:2:1: Error: Compiler version ^0.8.0 does not satisfy the r
semver requirement
VDA-V1.sol:26:53: Error: Use double quotes for string literals
VDA-V1.sol:143:51: Error: Use double quotes for string literals
VDA-V1.sol:151:51: Error: Use double quotes for string literals
VDA-V1.sol:159:42: Error: Use double quotes for string literals
VDA-V1.sol:160:35: Error: Use double quotes for string literals
VDA-V1.sol:171:41: Error: Use double quotes for string literals
```

VDALock.sol - now using Magna

```
VDA-Lock.sol:2:1: Error: Compiler version ^0.8.0 does not satisfy the
r semver requirement
VDA-Lock.sol:112:37: Error: Use double quotes for string literals
VDA-Lock.sol:118:21: Error: Avoid to make time-based decisions in
your business logic
VDA-Lock.sol:120:35: Error: Avoid to make time-based decisions in
your business logic
VDA-Lock.sol:152:21: Error: Avoid to make time-based decisions in
your business logic
VDA-Lock.sol:181:40: Error: Avoid to make time-based decisions in
your business logic
VDA-Lock.sol:185:14: Error: Avoid to make time-based decisions in
your business logic
VDA-Lock.sol:190:34: Error: Avoid to make time-based decisions in
your business logic
```

RecipientLockUp.sol - now using Magna

```
RecipientLockUp.sol:2:1: Error: Compiler version ^0.8.0 does not
satisfy the r semver requirement
RecipientLockUp.sol:96:37: Error: Use double quotes for string
literals
RecipientLockUp.sol:126:31: Error: Avoid to make time-based decisions
in your business logic
RecipientLockUp.sol:179:40: Error: Avoid to make time-based decisions
in your business logic
RecipientLockUp.sol:183:14: Error: Avoid to make time-based decisions
in your business logic
RecipientLockUp.sol:188:34: Error: Avoid to make time-based decisions
in your business logic
```

Software analysis result:

All results have been reviewed and can be safely ignored.



#Hashlock.

